

The Internet

- The internet connects millions of devices/hosts, they are connect with packets.
- To send info, it is first parsed through (packets), including header info
- Packets are then sent through the network to their destination using routers and link-layer switches.

Switch

Operates at the Data Link Layer (Layer 2).
Connects devices within the same network.
Uses MAC addresses for forwarding.

Router

Operates at the Network Layer (Layer 3).
Connects multiple different networks.
Uses IP addresses for routing.

Given the millions of ISPs, that exists, how do we connect them together?

- Tier-1 ISPs connect to each other through private peering points or Internet Exchange Points (IXPs) as a “network of networks.”
- Today: about a dozen Tier-1 ISPs
 - Ex: AT&T (AS7018), Level 3 (AS3356, AS3549), Deutsche Telekom (AS3320)
- Lower tiers (Tier-2/Tier-3) obtain transit by paying Tier-1 ISPs to access the rest of the internet.

These numbers are ASN, Autonomous System Number, it’s like an ID number for a network on the internet.

An Autonomous System (AS) is basically a large network controlled by one organization, like:

- an Internet provider
- a university
- a big company
- a cloud provider

You -> Regional ISP -> Global ISP

Examples of regional networks in the UK:

- British Telecom
- KCOM Group
- Vodafone

Protocols

- A protocol is like a language shared between two people. If you speak English and the other person speaks only French, communication fails. Similarly, two computers must agree on the same “language” (rules, timing, and format) to successfully exchange data.
- An RFC is a Request for Comments document. Many internet standards and protocol specifications are published as RFCs.
- Protocols can define addressing, message format, timing, error handling, ordering, security, and how endpoints start or end communication.

Internet Protocol Suite

- The internet protocol suite is the family of protocols used to build the internet.
- It is often called TCP/IP because TCP and IP are two of its core protocols.
- Important examples:
 1. Application: HTTP, DNS, FTP, SSH, Telnet.
 2. Transport: TCP, UDP.
 3. Network: IP, ICMP.
 4. Link-related support: Ethernet, Wi-Fi, ARP.

- The suite is layered so each layer can focus on a narrower job.

Five-layer Internet Protocol Stack

- Application Layer: Where network applications (web browsers, email) live (e.g., HTTP, SMTP).
- Transport Layer: Handles end-to-end data transfer and reliability (e.g., TCP, UDP).
- Network Layer: Manages packet routing and addressing across different networks (e.g., IP).
- Link Layer: Manages data transfer between neighboring network nodes (e.g., Ethernet, Wi-Fi).
- Physical Layer: Transmits raw bits over a physical medium (e.g., cables, fiber, radio waves).

Physical Media

- Guided media: signals that propagate in solid media. Ex: copper, fibre, co-ax
- Unguided media: signals that propagate freely. Ex: radio, microwave

Encapsulation

- Each layer adds its own header to the data it receives from the layer above.
- Names commonly used at each layer:
 1. Application data: message.
 2. Transport layer: segment for TCP, datagram for UDP.
 3. Network layer: packet or IP datagram.
 4. Link layer: frame.
 5. Physical layer: bits.
- Example when loading a web page:
 1. HTTP creates a request message.
 2. TCP wraps it in a segment with source/destination ports.
 3. IP wraps it in a packet with source/destination IP addresses.
 4. Ethernet or Wi-Fi wraps it in a frame with MAC addresses.
 5. The physical layer sends the bits over copper, fibre, or radio.
- Decapsulation happens in the reverse direction at the receiver.

Application Layer

- The application layer is where user-facing network applications and their protocols live.
- Examples:
 1. Web: HTTP and HTTPS.
 2. Email: SMTP, IMAP, POP3.
 3. File transfer: FTP, SFTP.
 4. Remote login: SSH, Telnet.
 5. Name lookup: DNS.
- A protocol defines:
 1. message types.
 2. message syntax and fields.
 3. message meaning.
 4. rules for when messages are sent and how receivers respond.

Client and Server Model

- A **server** is usually always on, has a stable address, and waits for requests.
- A **client** initiates communication with a server.
- Servers usually listen on well-known ports:
 1. HTTP: 80
 2. HTTPS: 443
 3. SSH: 22

- 4. DNS: 53
- 5. FTP command channel: 21
- Some applications are peer-to-peer, where hosts can act as both client and server.

HTTP

- HTTP is the main application protocol for the web.
- It is a request/response protocol:
 1. client sends an HTTP request.
 2. server sends an HTTP response.
- Common methods:
 1. GET: request a resource.
 2. POST: submit data to be processed.
 3. PUT: replace or create a resource.
 4. DELETE: remove a resource.
 5. HEAD: get headers without the response body.
- Status code groups:
 1. 1xx: informational.
 2. 2xx: success, such as 200 OK.
 3. 3xx: redirection.
 4. 4xx: client error, such as 404 Not Found.
 5. 5xx: server error, such as 500 Internal Server Error.
- Common HTTP status codes:

Code	Meaning
200 OK	Request succeeded.
301 Moved Permanently	Resource has a permanent new URL.
302 Found	Temporary redirect.
400 Bad Request	Client sent an invalid request.
401 Unauthorized	Authentication is required or failed.
403 Forbidden	Server understood the request but refuses access.
404 Not Found	Requested resource was not found.
500 Internal Server Error	Server failed while processing the request.
503 Service Unavailable	Server is temporarily unable to handle the request.

- HTTP is stateless: each request is independent unless the application adds state using cookies, sessions, tokens, or similar mechanisms.

Telnet

- Telnet is an older remote terminal protocol.
- It commonly uses TCP port 23.
- Telnet sends data, including login details, in plaintext.
- Because it is insecure, SSH is normally preferred for real remote access.
- Telnet is still useful in labs for manually testing text-based protocols such as HTTP:

```
telnet example.com 80
GET / HTTP/1.1
Host: example.com
```

Client/Server and Asynchronous Communication

- In a synchronous request/response flow, the client waits for the server's reply before continuing that task.
- In asynchronous communication, a program can start a network operation and continue doing other work while waiting for the response.
- Asynchronous designs are useful for servers because they may need to handle many clients at once without blocking on one slow connection.
- Common approaches include event loops, callbacks, futures/promises, and non-blocking sockets.

Transport Layer

- The transport layer provides communication between processes on hosts.
- It uses port numbers so multiple applications can share the same network interface.
- A socket is commonly identified by protocol, IP address, and port.

UDP

- UDP is connectionless.
- It has low overhead and does not establish a session before sending.
- It does not guarantee delivery, ordering, duplicate protection, or congestion control.
- It is useful for DNS, DHCP, streaming, games, and applications that prefer speed or implement their own reliability.

TCP

- TCP is connection-oriented.
- It provides reliable, ordered byte-stream delivery.
- Features:
 1. sequence numbers.
 2. acknowledgements.
 3. retransmission of lost data.
 4. flow control so a fast sender does not overwhelm a receiver.
 5. congestion control so traffic adapts to network load.
- TCP establishes connections using a three-way handshake:
 1. SYN
 2. SYN-ACK
 3. ACK
- TCP is used by HTTP, HTTPS, SSH, FTP, and many other reliable services.

Delay and Throughput

- Important performance measures:
 1. **Processing delay**: time spent examining packet headers and deciding what to do.
 2. **Queuing delay**: time waiting in router or switch buffers.
 3. **Transmission delay**: time needed to push all bits onto the link.
 4. **Propagation delay**: time for the signal to travel through the medium.
- Throughput is the rate at which useful data is delivered.
- Bottleneck links limit end-to-end throughput.